Bases de Datos

Ayudantía: Tarea 3

Optimización de consultas

Tarea 3: Objetivos

1. Leer **planes de ejecución** para identificar qué ralentiza una consulta.
2. Decidir qué **índices** y qué **vista materializada** crear.
3. **Reescribir** consultas SQL ineficientes.

Tarea 3: Meta

El workload de 15 consultas debe ejecutarse **al menos 8 veces más rápido** que en la línea base.

$$\text{speedup} = \frac{\text{tiempo total base}}{\text{tiempo total final}} \geq 8$$

Tarea 3: Partes

Parte	Tema
0	Preparar la BD y línea base
1	Clasificar consultas
2	Índices
3	Vista materializada
4	Reescritura de consultas

Tarea 3: Contexto

Una **librería** con ~2,4 millones de pedidos.

- `orders`, `order_items`
- `users`
- `books`, `authors`, `inventory`
- `reviews`

> Los ejemplos de hoy usan una BD de 500.000 pedidos con este mismo esquema. **No es la BD de la tarea.**

Setup: orders y users

```sql
CREATE TABLE users (
  user_id int PRIMARY KEY, email text, country text, is_premium boolean
);
CREATE TABLE orders (
  order_id int PRIMARY KEY, user_id int, order_date timestamp,
  status text, total_amount numeric(10,2)
);

INSERT INTO users
SELECT i, 'u' || i || '@mail.cl',
       (ARRAY['CL','AR','PE','MX','ES','CO','BR'])[1 + i % 7],
       random() < 0.15
FROM generate_series(1, 20000) AS i;
```

Setup: orders y users

```
INSERT INTO orders
SELECT i, 1 + (random() * 19999)::int,
       timestamp '2022-01-01' + (random() * 1460) * interval '1 day',
       (ARRAY['pending','shipped','delivered','cancelled','refunded'])[1 + i % 5],
       round((10 + random() * 390)::numeric, 2)
FROM generate_series(1, 500000) AS i;

ANALYZE users;
ANALYZE orders;
```

500.000 pedidos y 20.000 usuarios. Solo existen los índices de las PK.

Planes de ejecución

Plan de ejecución

- Un plan de `EXPLAIN` es un **árbol de operadores físicos**.
- Cada **nodo** es un algoritmo concreto: `Seq Scan`, `Hash Join`, `Sort`…
- Los hijos le entregan filas a su padre: se ejecuta de adentro hacia afuera.
- El plan **lógico** (clase 16) dice *qué* se calcula; el **físico** (clases 18 a 20) elige *cómo*.

EXPLAIN vs. EXPLAIN ANALYZE

Comando	¿Ejecuta la consulta?	¿Qué muestra?
`EXPLAIN`	No	Plan + costos **estimados**
`EXPLAIN ANALYZE`	**Sí**	+ tiempos y filas **reales**
`EXPLAIN (ANALYZE, BUFFERS)`	Sí	+ páginas de buffer vs. disco

EXPLAIN: solo el plan estimado

```
EXPLAIN SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
  ↳ Parallel Seq Scan on orders
       (cost=0.00..6480.17 rows=10 width=26)
       Filter: (user_id = 777)
```

- `Parallel Seq Scan`: el algoritmo elegido (un full scan repartido entre procesos).
- `cost=0.00..6480.17`: costo **estimado** (startup..total).
- `rows=10`: filas estimadas por proceso (`Gather` estima 25 en total).

→ El plan, pero **sin tocar la BD**.

Leer un plan: cada dato

```
EXPLAIN SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
  ↳ Parallel Seq Scan on orders
        (cost=0.00..6480.17 rows=10 width=26)
        Filter: (user_id = 777)
```

Nodo: cada línea con nombre es un algoritmo. `Parallel Seq Scan` es un *full scan* repartido entre procesos y `Gather` junta sus resultados. El hijo va indentado.

Algoritmos en EXPLAIN: lectura de tablas

Nodo	En clases	Qué hace
`Seq Scan`	Full scan	recorre toda la tabla
`Index Scan`	Selección con índice (B+tree o hash)	busca en el índice y va a la tupla por su RID
`Bitmap Heap Scan`	(no lo vimos)	el índice marca las páginas; las lee una vez, en orden

El índice por defecto es un **B+tree**; `USING hash` solo sirve para `=`. En PostgreSQL los índices son **unclustered**: la tabla no se ordena según el índice.

Algoritmos en EXPLAIN: joins y agregación

Nodo	En clases	Qué hace
`Nested Loop`	Nested Loops Join (o Index NLJ)	por cada fila externa, busca en la interna
`Hash Join`	Hash Join	join con tabla hash
`Merge Join`	Sort-Merge Join	join de dos entradas ordenadas
`HashAggregate`	GroupBy (γ) con hashing	agrupa con tabla hash
`SubPlan`	(no lo vimos)	subconsulta evaluada dentro de otro nodo

Leer un plan: cada dato

```
EXPLAIN SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
  ↳ Parallel Seq Scan on orders
       (cost=0.00..6480.17 rows=10 width=26)
       Filter: (user_id = 777)
```

Costo de inicio (*startup*): lo que cuesta antes de entregar la primera fila. `Gather` parte en 1000 (lanzar los procesos paralelos); un scan empieza de inmediato: 0.

Leer un plan: cada dato

```
EXPLAIN SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
  ↳ Parallel Seq Scan on orders
       (cost=0.00..6480.17 rows=10 width=26)
       Filter: (user_id = 777)
```

Costo total: hasta entregar la última fila. Es **acumulado**: los 7482.67 de `Gather` incluyen los 6480.17 de su hijo.

Costo estimado

- **Unidades internas** del optimizador, no milisegundos.
- Mezcla ponderada de **I/O y CPU**.
- Sirve para **comparar planes** de una misma consulta.

Parámetro	Valor	Qué mide
`seq_page_cost`	1,0	leer una página, secuencial
`random_page_cost`	4,0	leer una página, aleatoria
`cpu_tuple_cost`	0,01	procesar una fila
`cpu_operator_cost`	0,0025	evaluar un filtro

Costo de inicio y costo total

	Qué es
Startup (primer número)	costo **antes** de poder entregar la primera fila
Total (segundo número)	costo **hasta** entregar la última fila

Ambos son **acumulados**: incluyen el costo de los nodos hijos.

Startup y pipeline

Clases 17 y 20: pipeline contra materialización.

Startup no es *cuándo* empieza un nodo, sino **cuánto trabajo hay antes de su primera fila**.

- `Seq Scan`: entrega filas apenas las lee. Startup = **0**.
- `Sort`: no puede entregar ninguna fila hasta haber leído y ordenado **todo**. Ese trabajo cuenta como startup: **alto**.
- `Hash Join`: antes de la primera fila construye la tabla hash.

Leer un plan: cada dato

```
EXPLAIN SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
  ↳ Parallel Seq Scan on orders
       (cost=0.00..6480.17 rows=10 width=26)
       Filter: (user_id = 777)
```

Filas estimadas. `Gather` : 25 en total. El scan paralelo estima **por proceso**: 25 ÷ 2,4 ≈ 10.

Leer un plan: cada dato

```
EXPLAIN SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
  ↳ Parallel Seq Scan on orders
       (cost=0.00..6480.17 rows=10 width=26)
       Filter: (user_id = 777)
```

Ancho promedio de cada fila de salida, en bytes.

Leer un plan: cada dato

```
EXPLAIN SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
  ↳ Parallel Seq Scan on orders
       (cost=0.00..6480.17 rows=10 width=26)
       Filter: (user_id = 777)
```

Filter: condición evaluada dentro del scan; las filas que no cumplen se descartan.

EXPLAIN: no ejecuta

```
EXPLAIN SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
  ↳ Parallel Seq Scan on orders
       (cost=0.00..6480.17 rows=10 width=26)
       Filter: (user_id = 777)
```

Solo valores **estimados**: no hay `actual time`, ni `loops`, ni `Execution Time`.

EXPLAIN ANALYZE: ejecuta y mide

```
EXPLAIN ANALYZE SELECT order_id, order_date, status, total_amount
FROM orders WHERE user_id = 777;
```

```
Gather  (cost=1000.00..7482.67 rows=25 width=26)
        (actual time=0.464..10.602 rows=25.00 loops=1)
  ↳ Parallel Seq Scan on orders
      (cost=0.00..6480.17 rows=10 width=26)
      (actual time=0.323..6.592 rows=8.33 loops=3)
      Filter: (user_id = 777)
      Rows Removed by Filter: 166658
Execution Time: 10.645 ms
```

Campos nuevos

Campo	Significado
`actual time`	tiempo real (ms) **por loop**
`rows=8.33`	filas reales por loop
`loops=3`	veces que se ejecutó el nodo
`Rows Removed by Filter`	filas leídas y descartadas

Si `loops` > 1, el nodo se ejecutó varias veces. Se explica en la siguiente slide.

loops en un Nested Loop

Nested Loops Join (clase 19):

```
for r in R:                    # lado externo: se recorre 1 vez
    for s in S:                # lado interno: se recorre por cada r
        if cumple(r, s): resultado.append((r, s))
```

- El lado interno se ejecuta **una vez por cada fila** del externo: `loops` = filas del externo.
- Un `SubPlan` que depende de la fila de afuera funciona igual.
- En un `Parallel Seq Scan`, `loops=3` son 3 procesos que se reparten la tabla.

EXPLAIN (ANALYZE, BUFFERS)

`shared hit`: página ya en el buffer pool (RAM). `shared read`: se pidió fuera del buffer pool (típicamente, disco).

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, order_date, status, total_amount FROM orders WHERE user_id = 777;
```

```
Gather  (actual time=0.787..11.789 rows=25.00 loops=1)
  Buffers: shared read=3876
  ↳ Parallel Seq Scan on orders  (actual time=1.435..7.925 rows=8.33 loops=3)
       Filter: (user_id = 777)
       Buffers: shared read=3876
Execution Time: 11.834 ms
```

Sin índice se leen las **3.876 páginas** de la tabla.

EXPLAIN (ANALYZE, BUFFERS): con índice

```
CREATE INDEX idx_ord_user ON orders (user_id);

EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, order_date, status, total_amount FROM orders WHERE user_id = 777;

DROP INDEX idx_ord_user;   -- para los ejemplos siguientes
```

```
Bitmap Heap Scan on orders  (actual time=0.044..0.160 rows=25.00 loops=1)
  Buffers: shared hit=25 read=3
  ↳ Bitmap Index Scan on idx_ord_user  (actual rows=25.00 loops=1)
        Buffers: shared read=3
Execution Time: 0.199 ms
```

Con el índice se tocan **28 páginas** (25 `hit` + 3 `read`) en vez de 3.876: es el I/O de las clases 12 y 13.

Cuidado con escrituras

`EXPLAIN ANALYZE` con `INSERT` / `UPDATE` / `DELETE` **sí ejecuta** la operación.

```
BEGIN;
EXPLAIN ANALYZE DELETE FROM orders WHERE status = 'cancelled';
ROLLBACK;
```

Índices

La consulta de referencia

```
EXPLAIN ANALYZE
SELECT o.order_id, u.email, o.total_amount
FROM orders o JOIN users u ON o.user_id = u.user_id
WHERE o.total_amount > 290 AND u.country = 'CL';
```

La iremos corriendo después de cada cambio de índices y comparando los planes.

Paso 1: sin índices adicionales

```
Gather
  ↳ Hash Join
       Hash Cond: (o.user_id = u.user_id)
       ↳ Parallel Seq Scan on orders o
            Filter: (total_amount > 290)
       ↳ Hash
            ↳ Seq Scan on users u
                 Filter: (country = 'CL')
Execution Time: 22.196 ms
```

Preguntas

1. ¿Qué tipo de join eligió PostgreSQL?

2. ¿Dónde aplicó las selecciones (`total_amount > 290`, `country = 'CL'`)?

3. ¿Usó algún índice?

Paso 1: sin índices adicionales

```
Gather
  ↳ Hash Join
        Hash Cond: (o.user_id = u.user_id)
        ↳ Parallel Seq Scan on orders o
              Filter: (total_amount > 290)
        ↳ Hash
              ↳ Seq Scan on users u
                    Filter: (country = 'CL')
Execution Time: 22.196 ms
```

Preguntas

1. ¿Qué tipo de join eligió PostgreSQL?
   → `Hash Join`: arma una tabla hash con los usuarios filtrados y recorre `orders`.
2. ¿Dónde aplicó las selecciones (`total_amount > 290`, `country = 'CL'`)?

3. ¿Usó algún índice?

Paso 1: sin índices adicionales

```
Gather
  ↳ Hash Join
        Hash Cond: (o.user_id = u.user_id)
        ↳ Parallel Seq Scan on orders o
              Filter: (total_amount > 290)
        ↳ Hash
              ↳ Seq Scan on users u
                    Filter: (country = 'CL')
Execution Time: 22.196 ms
```

Preguntas

1. ¿Qué tipo de join eligió PostgreSQL?
   → `Hash Join`: arma una tabla hash con los usuarios filtrados y recorre `orders`.
2. ¿Dónde aplicó las selecciones (`total_amount > 290`, `country = 'CL'`)?
   → Antes del join, en cada `Seq Scan` (`Filter:`): push-selection (clase 16).
3. ¿Usó algún índice?

Paso 1: sin índices adicionales

```
Gather
  ↳ Hash Join
        Hash Cond: (o.user_id = u.user_id)
        ↳ Parallel Seq Scan on orders o
              Filter: (total_amount > 290)
        ↳ Hash
              ↳ Seq Scan on users u
                    Filter: (country = 'CL')
Execution Time: 22.196 ms
```

Preguntas

1. ¿Qué tipo de join eligió PostgreSQL?
   → `Hash Join`: arma una tabla hash con los usuarios filtrados y recorre `orders`.
2. ¿Dónde aplicó las selecciones (`total_amount > 290`, `country = 'CL'`)?
   → Antes del join, en cada `Seq Scan` (`Filter:`): push-selection (clase 16).
3. ¿Usó algún índice?
   → No: las tablas se leen con `Seq Scan` y no hay nodos `Index Scan` ni `Bitmap Index Scan`.

Paso 1: sin índices adicionales

```
Gather  (cost=1413.71..10105.61 rows=20544 width=24)
        (actual time=2.568..21.681 rows=20086.00 loops=1)
  ↳ Hash Join  (cost=413.71..7051.21 rows=8560 width=24)
                 (actual time=1.402..15.837 rows=6695.33 loops=3)
       Hash Cond: (o.user_id = u.user_id)
       ↳ Parallel Seq Scan on orders o
            (cost=0.00..6480.17 rows=59924 width=14)
            (actual time=0.008..11.829 rows=47162.00 loops=3)
            Filter: (total_amount > 290)
       ↳ Hash  (cost=378.00..378.00 rows=2857 width=18)
            ↳ Seq Scan on users u
                 (cost=0.00..378.00 rows=2857 width=18)
                 (actual time=0.008..1.057 rows=2857.00 loops=3)
                 Filter: (country = 'CL')
Execution Time: 22.196 ms
```

Preguntas

4. ¿Cuántas filas estimó y cuántas produjo realmente?

Paso 1: sin índices adicionales

```
Gather  (cost=1413.71..10105.61 rows=20544 width=24)
        (actual time=2.568..21.681 rows=20086.00 loops=1)
  ↳ Hash Join  (cost=413.71..7051.21 rows=8560 width=24)
                 (actual time=1.402..15.837 rows=6695.33 loops=3)
       Hash Cond: (o.user_id = u.user_id)
       ↳ Parallel Seq Scan on orders o
            (cost=0.00..6480.17 rows=59924 width=14)
            (actual time=0.008..11.829 rows=47162.00 loops=3)
            Filter: (total_amount > 290)
       ↳ Hash  (cost=378.00..378.00 rows=2857 width=18)
            ↳ Seq Scan on users u
                 (cost=0.00..378.00 rows=2857 width=18)
                 (actual time=0.008..1.057 rows=2857.00 loops=3)
                 Filter: (country = 'CL')
Execution Time: 22.196 ms
```

Preguntas

4. ¿Cuántas filas estimó y cuántas produjo realmente?

   → En `Gather` : estimó 20.544 y salieron 20.086. La estimación es muy cercana.

Paso 2: índice en total_amount

```
CREATE INDEX idx_ord_total ON orders (total_amount);
-- se vuelve a correr la consulta de referencia
```

```
Hash Join  (actual time=5.065..37.190 rows=20086.00 loops=1)
  ↳ Bitmap Heap Scan on orders o  (rows=141486.00 loops=1)
       Heap Blocks: exact=3876
       ↳ Bitmap Index Scan on idx_ord_total
            Index Cond: (total_amount > 290)
  ↳ Hash
       ↳ Seq Scan on users u
Execution Time: 37.834 ms
```

Preguntas

1. ¿Cambió el plan? ¿Usa el nuevo índice?

Paso 2: índice en total_amount

```
CREATE INDEX idx_ord_total ON orders (total_amount);
-- se vuelve a correr la consulta de referencia
```

```
Hash Join  (actual time=5.065..37.190 rows=20086.00 loops=1)
  ↳ Bitmap Heap Scan on orders o  (rows=141486.00 loops=1)
        Heap Blocks: exact=3876
        ↳ Bitmap Index Scan on idx_ord_total
              Index Cond: (total_amount > 290)
  ↳ Hash
        ↳ Seq Scan on users u
Execution Time: 37.834 ms
```

Preguntas

1. ¿Cambió el plan? ¿Usa el nuevo índice?

   → Sí: lee `orders` con un `Bitmap Index Scan` sobre `idx_ord_total`. El join sigue siendo `Hash Join`.

Paso 2: índice en total_amount

```
CREATE INDEX idx_ord_total ON orders (total_amount);
-- se vuelve a correr la consulta de referencia
```

```
Hash Join  (actual time=5.065..37.190 rows=20086.00 loops=1)
  ↳ Bitmap Heap Scan on orders o  (rows=141486.00 loops=1)
        Heap Blocks: exact=3876
        ↳ Bitmap Index Scan on idx_ord_total
              Index Cond: (total_amount > 290)
  ↳ Hash
        ↳ Seq Scan on users u
Execution Time: 37.834 ms
```

Preguntas

2. ¿Mejoró el tiempo (22,2 ms antes)?

Paso 2: índice en total_amount

```
CREATE INDEX idx_ord_total ON orders (total_amount);
-- se vuelve a correr la consulta de referencia
```

```
Hash Join  (actual time=5.065..37.190 rows=20086.00 loops=1)
  ↳ Bitmap Heap Scan on orders o  (rows=141486.00 loops=1)
        Heap Blocks: exact=3876
        ↳ Bitmap Index Scan on idx_ord_total
              Index Cond: (total_amount > 290)
  ↳ Hash
        ↳ Seq Scan on users u
Execution Time: 37.834 ms
```

Preguntas

2. ¿Mejoró el tiempo (22,2 ms antes)?

   → No: 37,8 ms. Pasan 141.486 de las 500.000 filas (28 %) y se leen las 3.876 páginas de la tabla (`Heap Blocks`), además de recorrer el índice.

¿Cuánto filtra realmente total_amount > 290?

```
SELECT COUNT(*) FILTER (WHERE total_amount > 290) AS sobre_290,
       COUNT(*)                                   AS total,
       ROUND(100.0 * COUNT(*) FILTER (WHERE total_amount > 290)
             / COUNT(*), 1)                       AS pct
FROM orders;
```

sobre_290	total	pct
141486	500000	28,3

→ El **28 %** de las filas pasan el filtro `total_amount > 290` (141.486 de 500.000): casi un tercio de la tabla. El índice ahorra poco, y con él la consulta tardó **más**.

Paso 3: índice en user_id

```
CREATE INDEX idx_ord_user ON orders (user_id);
-- se vuelve a correr la consulta de referencia
```

```
Hash Join  (actual time=5.076..37.272 rows=20086.00 loops=1)
  Hash Cond: (o.user_id = u.user_id)
  ↳ Bitmap Heap Scan on orders o  (rows=141486.00 loops=1)
        ↳ Bitmap Index Scan on idx_ord_total
  ↳ Hash
        ↳ Seq Scan on users u
Execution Time: 37.852 ms
```

Preguntas

1. ¿Usa el nuevo índice (Index Nested Loops Join) o sigue con Hash Join? ¿Por qué?

Paso 3: índice en user_id

```
CREATE INDEX idx_ord_user ON orders (user_id);
-- se vuelve a correr la consulta de referencia
```

```
Hash Join  (actual time=5.076..37.272 rows=20086.00 loops=1)
  Hash Cond: (o.user_id = u.user_id)
  ↳ Bitmap Heap Scan on orders o  (rows=141486.00 loops=1)
        ↳ Bitmap Index Scan on idx_ord_total
  ↳ Hash
        ↳ Seq Scan on users u
Execution Time: 37.852 ms
```

Preguntas

1. ¿Usa el nuevo índice (Index Nested Loops Join) o sigue con Hash Join? ¿Por qué?
   → Sigue con `Hash Join` y no usa `idx_ord_user`. El **por qué** queda como ejercicio.

Umbral de selectividad

¿Qué pasa con un filtro más o menos selectivo? (con `idx_ord_total` creado)

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE total_amount > 200;   -- ~51 % de las filas
```

```
Seq Scan on orders  (actual rows=257017.00)
Execution Time: 42.275 ms
```

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE total_amount > 361;   -- ~10 % de las filas
```

```
Bitmap Heap Scan on orders  (actual rows=50201.00)
Execution Time: 13.377 ms
```

El optimizador elige entre índice y *full scan* según sus costos estimados, y no siempre acierta (en el Paso 2, con 28 %, eligió el índice y salió peor). Con `SELECT *` el cambio ocurrió entre ~28 % y ~51 %.

Tamaño de tablas e índices

```
SELECT relname, pg_size_pretty(pg_relation_size(oid)) AS tamano
FROM pg_class
WHERE relname IN ('orders','orders_pkey','idx_ord_total','idx_ord_user');
```

relname	tamaño
`orders`	30 MB
`orders_pkey`	11 MB
`idx_ord_total`	11 MB
`idx_ord_user`	3,7 MB

Los índices **no son gratis**: ocupan disco y RAM en el buffer pool.

Filtrar y ordenar: el problema

```
DROP INDEX idx_ord_total;   -- sin idx_ord_total
EXPLAIN ANALYZE
SELECT order_id, user_id, total_amount FROM orders
WHERE status = 'pending' ORDER BY total_amount DESC LIMIT 10;
```

```
Limit  (actual time=14.389..16.808 rows=10.00 loops=1)
  ↳ Sort  (actual rows=7.67 loops=3)
        Sort Key: total_amount DESC
        ↳ Parallel Seq Scan on orders  (actual rows=33333.33 loops=3)
              Filter: (status = 'pending')
              Rows Removed by Filter: 133333
Execution Time: 16.822 ms
```

Plan abreviado: se omiten nodos intermedios (p. ej. `Gather Merge`).

Preguntas

1. ¿Cuántas filas lee y cuántas devuelve?
2. ¿Bastaría un índice sobre `status`?

Filtrar y ordenar: índice compuesto

```
CREATE INDEX idx_ord_status_total ON orders (status, total_amount);
```

```
Limit  (actual time=0.022..0.031 rows=10.00 loops=1)
  ↳ Index Scan Backward using idx_ord_status_total on orders
        (actual rows=10.00 loops=1)
        Index Cond: (status = 'pending')
Execution Time: 0.041 ms
```

Preguntas

1. ¿Por qué desapareció el `Sort`?
2. ¿Serviría un índice Hash?

Estadísticas y optimizador

ANALYZE tabla

```
ANALYZE orders;
```

- Lee una **muestra** de la tabla y guarda sus estadísticas en el catálogo del sistema (clase 16).
- Número de filas y páginas, valores distintos, histogramas.
- El optimizador las usa para **estimar** `rows` y elegir el plan (clase 20).
- `ANALYZE;` sin tabla actualiza toda la BD.

Estadísticas desactualizadas

¿Qué pasa cuando los datos cambian y el optimizador no se entera?

```
CREATE TABLE orders_demo AS SELECT * FROM orders;   -- 500.000 filas
CREATE INDEX ON orders_demo (order_date);
ALTER TABLE orders_demo SET (autovacuum_enabled = false);
ANALYZE orders_demo;

-- 400.000 pedidos nuevos, todos de 2026
INSERT INTO orders_demo
SELECT 500000 + i, 1 + (random() * 19999)::int,
       timestamp '2026-01-01' + (random() * 300) * interval '1 day',
       'pending', 100
FROM generate_series(1, 400000) AS i;
```

autovacuum: proceso que ejecuta `ANALYZE` solo, cuando cambió suficiente de la tabla. Lo desactivamos para el ejemplo.

Sin ANALYZE: las estimaciones mienten

```
EXPLAIN ANALYZE SELECT * FROM orders_demo WHERE order_date >= '2026-01-01';
```

```
Bitmap Heap Scan on orders_demo
    (cost=173.16..7479.72 rows=8353 width=30)
    (actual time=14.871..33.366 rows=400000.00 loops=1)
  Recheck Cond: (order_date >= '2026-01-01')
  ↳ Bitmap Index Scan on orders_demo_order_date_idx
Execution Time: 42.434 ms
```

Estimó **8.353** filas y salieron **400.000**: el optimizador usa las estadísticas viejas.

Después de ANALYZE

```
ANALYZE orders_demo;
SELECT reltuples::int FROM pg_class WHERE relname = 'orders_demo';   -- 900000 (antes: 500000)
EXPLAIN ANALYZE SELECT * FROM orders_demo WHERE order_date >= '2026-01-01';
```

```
Seq Scan on orders_demo
    (cost=0.00..18095.00 rows=399287 width=30)
    (actual time=20.662..46.710 rows=400000.00 loops=1)
  Filter: (order_date >= '2026-01-01')
  Rows Removed by Filter: 500000
Execution Time: 55.745 ms
```

El plan cambió y la estimación (399.287) ya se acerca a lo real. No siempre es más rápido: lo que cambia es que el optimizador decide con datos correctos.

Forzar un plan diferente (opcional)

PostgreSQL permite desactivar algoritmos para comparar planes:

```
SET enable_hashjoin = off;
EXPLAIN ANALYZE
SELECT u.country, SUM(o.total_amount)
FROM orders o JOIN users u ON o.user_id = u.user_id
GROUP BY u.country;
RESET enable_hashjoin;   -- restaurar
```

Solo para aprender. En la práctica, se deja que el optimizador decida.

Forzar un plan: resultado

```
-- por defecto
Hash Join  (actual time=2.997..29.671 rows=166666.67 loops=3)
  Hash Cond: (o.user_id = u.user_id)
Execution Time: 52.564 ms
```

```
-- con enable_hashjoin = off
Nested Loop  (actual time=0.347..74.091 rows=166666.67 loops=3)
  ↳ Parallel Seq Scan on orders o
  ↳ Memoize  (actual rows=1.00 loops=500000)
        ↳ Index Scan using users_pkey on users u  (actual rows=1.00 loops=59985)
Execution Time: 99.048 ms
```

Sin Hash Join eligió un Index Nested Loops Join (clase 19): el lado interno se consultó una vez por cada fila de `orders` (`Memoize`, `loops=500000`), pero solo buscó en el índice cuando el resultado no estaba en su caché (`loops=59985`).

Vistas materializadas

Vista materializada

```
CREATE MATERIALIZED VIEW libros_por_autor AS
SELECT author_id, COUNT(*) AS n_books
FROM books
GROUP BY author_id;
```

Guarda el resultado **en disco**, como una tabla.

Beneficios de una vista

- Leer el resultado cuesta casi nada: se lee como una **tabla**.
- El trabajo pesado (join y agregación) se hace **una sola vez**.
- Varias consultas pueden **reutilizar** la misma vista.
- Se puede crear un **índice** sobre ella.

Costos de una vista

- Se calcula **una vez**, al crearla.
- **No se actualiza sola** (investigar `TRIGGERS`).
- Ocupa disco.

```
REFRESH MATERIALIZED VIEW libros_por_autor;
```

Ejemplo: consulta agregada

```
EXPLAIN ANALYZE
SELECT u.country, SUM(o.total_amount) AS ventas
FROM orders o JOIN users u ON o.user_id = u.user_id
GROUP BY u.country;
```

```
Finalize GroupAggregate  (actual time=50.227..54.796 rows=7.00 loops=1)
  ↳ Partial HashAggregate  (actual rows=7.00 loops=3)
        ↳ Hash Join  (actual rows=166666.67 loops=3)
Execution Time: 54.951 ms
```

Plan abreviado: se omiten nodos intermedios (`Gather Merge` , `Sort`).

Ejemplo: leer la vista

```
CREATE MATERIALIZED VIEW ventas_pais AS
SELECT u.country, SUM(o.total_amount) AS ventas
FROM orders o JOIN users u ON o.user_id = u.user_id
GROUP BY u.country;

EXPLAIN ANALYZE SELECT country, ventas FROM ventas_pais;
```

```
Seq Scan on ventas_pais  (actual rows=7.00 loops=1)
Execution Time: 0.028 ms
```

Preguntas

1. ¿Cuántas filas entran al `GROUP BY` y cuántas salen?

2. ¿Habría ayudado un índice?

Ejemplo: leer la vista

```
CREATE MATERIALIZED VIEW ventas_pais AS
SELECT u.country, SUM(o.total_amount) AS ventas
FROM orders o JOIN users u ON o.user_id = u.user_id
GROUP BY u.country;

EXPLAIN ANALYZE SELECT country, ventas FROM ventas_pais;
```

```
Seq Scan on ventas_pais  (actual rows=7.00 loops=1)
Execution Time: 0.028 ms
```

Preguntas

1. ¿Cuántas filas entran al `GROUP BY` y cuántas salen?

   → Entran 500.000 (el `Hash Join` de la slide anterior: 166.666,67 × 3 loops) y salen 7: un grupo por país.

2. ¿Habría ayudado un índice?

Ejemplo: leer la vista

```
CREATE MATERIALIZED VIEW ventas_pais AS
SELECT u.country, SUM(o.total_amount) AS ventas
FROM orders o JOIN users u ON o.user_id = u.user_id
GROUP BY u.country;

EXPLAIN ANALYZE SELECT country, ventas FROM ventas_pais;
```

```
Seq Scan on ventas_pais  (actual rows=7.00 loops=1)
Execution Time: 0.028 ms
```

Preguntas

1. ¿Cuántas filas entran al `GROUP BY` y cuántas salen?

   → Entran 500.000 (el `Hash Join` de la slide anterior: 166.666,67 × 3 loops) y salen 7: un grupo por país.

2. ¿Habría ayudado un índice?

   → No ayuda mucho: igual hay que recorrer todas las filas para sumarlas. Por eso conviene guardar el resultado.

Reescritura de consultas

Consultas equivalentes

> Q y Q′ son **equivalentes** si Q(D) = Q′(D) **para toda base de datos D**.

Reescribir

- Buscar una Q′ equivalente a Q y **más eficiente** (clase 16).
- Las reglas del álgebra dan equivalencias demostrables.
- Comparar resultados en una BD **no demuestra** equivalencia.

Ejemplo: subconsulta correlacionada

```
DROP INDEX idx_ord_user;   -- sin índice en orders(user_id)
EXPLAIN ANALYZE SELECT u.user_id,
       (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.user_id) AS n
FROM users u WHERE u.country = 'CL';
```

```
Seq Scan on users u  (actual time=18.134..44081.773 rows=2857.00 loops=1)
  Filter: (country = 'CL')
  SubPlan 1
    ↳ Aggregate  (actual rows=1.00 loops=2857)
          ↳ Seq Scan on orders o  (actual rows=24.92 loops=2857)
                Filter: (user_id = u.user_id)
                Rows Removed by Filter: 499975
Execution Time: 44083.268 ms
```

Preguntas para discutir

1. ¿Cuántas veces se lee `orders`?

Ejemplo: subconsulta correlacionada

```
DROP INDEX idx_ord_user;   -- sin índice en orders(user_id)
EXPLAIN ANALYZE SELECT u.user_id,
       (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.user_id) AS n
FROM users u WHERE u.country = 'CL';
```

```
Seq Scan on users u  (actual time=18.134..44081.773 rows=2857.00 loops=1)
  Filter: (country = 'CL')
  SubPlan 1
    ↳ Aggregate  (actual rows=1.00 loops=2857)
          ↳ Seq Scan on orders o  (actual rows=24.92 loops=2857)
                Filter: (user_id = u.user_id)
                Rows Removed by Filter: 499975
Execution Time: 44083.268 ms
```

Preguntas para discutir

1. ¿Cuántas veces se lee `orders`?

   → 2.857 veces (`loops=2857`), una por usuario de CL; cada vez descarta 499.975 filas.

Ejemplo: subconsulta correlacionada

```
DROP INDEX idx_ord_user;   -- sin índice en orders(user_id)
EXPLAIN ANALYZE SELECT u.user_id,
       (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.user_id) AS n
FROM users u WHERE u.country = 'CL';
```

Preguntas para discutir

2. ¿Qué pasa con un usuario sin pedidos?

3. ¿Cómo leerla una sola vez?

Ejemplo: subconsulta correlacionada

```
DROP INDEX idx_ord_user;   -- sin índice en orders(user_id)
EXPLAIN ANALYZE SELECT u.user_id,
       (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.user_id) AS n
FROM users u WHERE u.country = 'CL';
```

Preguntas para discutir

2. ¿Qué pasa con un usuario sin pedidos?
   → `COUNT(*)` devuelve 0: el usuario aparece con `n = 0`.
3. ¿Cómo leerla una sola vez?

Ejemplo: subconsulta correlacionada

```
DROP INDEX idx_ord_user;   -- sin índice en orders(user_id)
EXPLAIN ANALYZE SELECT u.user_id,
       (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.user_id) AS n
FROM users u WHERE u.country = 'CL';
```

Preguntas para discutir

2. ¿Qué pasa con un usuario sin pedidos?
   → `COUNT(*)` devuelve 0: el usuario aparece con `n = 0`.
3. ¿Cómo leerla una sola vez?
   → Con un `LEFT JOIN` y `GROUP BY` (siguiente slide).

Ejemplo: una consulta equivalente

```sql
-- sin índice en orders(user_id)
EXPLAIN ANALYZE
SELECT u.user_id, COUNT(o.order_id) AS n
FROM users u LEFT JOIN orders o ON o.user_id = u.user_id
WHERE u.country = 'CL' GROUP BY u.user_id;
```

```
HashAggregate  (actual time=61.287..61.437 rows=2857.00 loops=1)
  ↳ Hash Right Join  (actual rows=71200.00 loops=1)
       ↳ Seq Scan on orders o  (actual rows=500000.00 loops=1)
       ↳ Hash
            ↳ Seq Scan on users u  (actual rows=2857.00 loops=1)
Execution Time: 61.598 ms
```

Preguntas para discutir

1. ¿Por qué mejoró (44.083 ms → 61,6 ms)?

Ejemplo: una consulta equivalente

```
-- sin índice en orders(user_id)
EXPLAIN ANALYZE
SELECT u.user_id, COUNT(o.order_id) AS n
FROM users u LEFT JOIN orders o ON o.user_id = u.user_id
WHERE u.country = 'CL' GROUP BY u.user_id;
```

```
HashAggregate  (actual time=61.287..61.437 rows=2857.00 loops=1)
  ↳ Hash Right Join  (actual rows=71200.00 loops=1)
       ↳ Seq Scan on orders o  (actual rows=500000.00 loops=1)
       ↳ Hash
            ↳ Seq Scan on users u  (actual rows=2857.00 loops=1)
Execution Time: 61.598 ms
```

Preguntas para discutir

1. ¿Por qué mejoró (44.083 ms → 61,6 ms)?
   → Ahora `orders` se lee una sola vez (`loops=1`), no 2.857 veces.

Ejemplo: una consulta equivalente

```sql
-- sin índice en orders(user_id)
EXPLAIN ANALYZE
SELECT u.user_id, COUNT(o.order_id) AS n
FROM users u LEFT JOIN orders o ON o.user_id = u.user_id
WHERE u.country = 'CL' GROUP BY u.user_id;
```

Preguntas para discutir

2. ¿Por qué `LEFT JOIN` y `COUNT(o.order_id)`?

Ejemplo: una consulta equivalente

```sql
-- sin índice en orders(user_id)
EXPLAIN ANALYZE
SELECT u.user_id, COUNT(o.order_id) AS n
FROM users u LEFT JOIN orders o ON o.user_id = u.user_id
WHERE u.country = 'CL' GROUP BY u.user_id;
```

Preguntas para discutir

2. ¿Por qué `LEFT JOIN` y `COUNT(o.order_id)`?

   → El `LEFT JOIN` conserva a los usuarios sin pedidos y `COUNT(o.order_id)` no cuenta los `NULL`, dando 0. Con `JOIN` o `COUNT(*)` el resultado cambiaría.

Herramientas

Otras herramientas útiles

Herramienta	Para qué sirve
`\timing` en psql	medir el tiempo total de una query
`EXPLAIN (ANALYZE, BUFFERS)`	plan + tiempos + páginas RAM/disco
`pg_stat_statements`	ranking de queries más lentas o frecuentes
`pg_stat_user_indexes`	índices que nunca se usan
`ANALYZE tabla`	actualizar estadísticas
`VACUUM`	marcar como reutilizable el espacio de tuplas muertas

Lecciones

Observación	Lección
El índice no siempre se usa	Depende de la selectividad
Una consulta lenta no siempre se arregla con un índice	A veces hay que reescribirla
Un índice tiene costo	Disco y escrituras
Las estimaciones pueden fallar	`ANALYZE` mantiene las estadísticas al día
Un agregado grande	Se puede guardar el resultado

Preguntas